OGC® DOCUMENT: 26-044

External identifier of this OGC® document: http://www.opengis.net/doc/IS/indoorsql/1.0

Open Geospatial Consortium

OGC INDOORGML 2.0 PART 2C – SQL ENCODING

STANDARD
Implementation

CANDIDATE SWG DRAFT

Version: 0.0.9
Submission Date: 2026-06-18
Approval Date: 2026-XX-XX
Publication Date: 2026-XX-XX
Editor: Sisi Zlatanova, Zhiyong Wang, Taehoon Kim, Abdoulaye Diakite, Ki-Joune Li

Notice for Drafts: This document is not an OGC Standard. This document is distributed for review and comment. This document is subject to change without notice and may not be referred to as an OGC Standard.
Recipients of this document are invited to submit, with their comments, notification of any relevant patent rights of which they are aware and to provide supporting documentation.

License Agreement

Use of this document is subject to the license agreement at https://www.ogc.org/license

Suggested additions, changes and comments on this document are welcome and encouraged. Such suggestions may be submitted using the online change request form on OGC web site: http://ogc.standardstracker.org/

Copyright notice

Copyright © 2026 Open Geospatial Consortium
To obtain additional rights of use, visit https://www.ogc.org/legal

Note

Attention is drawn to the possibility that some of the elements of this document may be the subject of patent rights. The Open Geospatial Consortium shall not be held responsible for identifying any or all such patent rights.

Recipients of this document are requested to submit, with their comments, notification of any relevant patent claims or other intellectual property rights of which they may be aware that might be infringed by any implementation of the standard set forth in this document, and to provide supporting documentation.

CONTENTS

I. ABSTRACT v

II. KEYWORDS v

III. PREFACE vi

IV. SECURITY CONSIDERATIONS vii

V. SUBMITTING ORGANIZATIONS viii

VI. SUBMITTERS viii

1. SCOPE 2

2. CONFORMANCE 4

3. NORMATIVE REFERENCES 6

4. TERMS AND DEFINITIONS 8

5. CONVENTIONS 11
   - 5.1. Symbols (and abbreviated terms) 11
   - 5.2. Identifiers 11
   - 5.3. Use of SQL terminology 11
   - 5.4. Geometry storage 12
   - 5.5. Complex DataType storage 12
   - 5.6. Code list storage 13
   - 5.7. Graph and route associations 13
   - 5.8. Table and column naming 13

6. OVERVIEW OF THE SQL ENCODING 15
   - 6.1. Design Principle 15

7. SQL ENCODING OF THE CORE MODULE 18
   - 7.1. Core code list types 18
   - 7.2. IndoorFeatures 19
   - 7.3. ThematicLayer 20
   - 7.4. PrimalSpaceLayer 21
   - 7.5. CellSpace 22
   - 7.6. CellBoundary 23

7.7. DualSpaceLayer 24
7.8. Node 25
7.9. Edge 26
7.10. InterLayerConnection 27
7.11. ExternalReferenceType 29

8. SQL ENCODING OF THE NAVIGATION MODULE 32
8.1. Navigation code list types 32
8.2. NavigableSpace 33
8.3. GeneralSpace 34
8.4. TransferSpace 34
8.5. NonNavigableSpace 35
8.6. ObjectSpace 36
8.7. NonNavigableBoundary 36
8.8. NavigableBoundary 37
8.9. Route 37

ANNEX A (NORMATIVE) CONFORMANCE CLASS ABSTRACT TEST SUITE 40
A.1. Conformance Class Core 40
A.2. Conformance Class Navigation 40

ANNEX B (NORMATIVE) SCHEMAS 42
B.1. SQL Schema of IndoorGML Core 42
B.2. SQL Schema of IndoorGML Navigation 48

ANNEX C (INFORMATIVE) REVISION HISTORY 54

BIBLIOGRAPHY 56

I ABSTRACT

The IndoorGML 2.0 Part 2c – SQL Encoding Standard presents the implementation-dependent Structured Query Language (SQL) encoding of the concepts defined by the IndoorGML 2.0 Part 1 – Conceptual Model Standard (OGC 22-045r5). The SQL Encoding is compliant with SQL as specified by ISO/IEC 9075 and is implemented using PostgreSQL as the reference database management system. The SQL encoding of IndoorGML 2.0 defines a concrete implementation schema for representing indoor spatial information, including primal space, dual space, thematic layers, interlayer connections, and navigation-related indoor features. This encoding can be used to store and exchange 2D and/or 3D indoor space and indoor navigation network models in relational databases as data sets or via web services.

This Part 2c of the Standard defines how the concepts developed in Part 1 are realized using SQL table definitions. The SQL encoding represents a mapping of the Conceptual Model derived from the UML model using Enterprise Architecture tooling, with subsequent corrections to identifier types, relationships, and missing properties. Geometry values are stored in PostGIS `geometry` columns. Geometry DataTypes from the conceptual model are flattened onto owning feature tables using path-style column names (for example, `cellSpaceGeom_geometry2D`), and non-geometric reference DataTypes are encoded as PostgreSQL composite types. Table 1 maps requirements classes from the IndoorGML 2.0 Conceptual Model into the implementation details documented in this standard.

Table 1

CONCEPTUAL MODEL	SECTION	SQL SCHEMA
IndoorGML Core	Clause 7	Annex B.1
Navigation Extension	Clause 8	Annex B.2

II KEYWORDS

The following are keywords to be used by search engines and document catalogues.

OGCDoc, OGC documents, indoor, navigation, IndoorGML, SQL, PostgreSQL, PostGIS

PREFACE

In order to achieve consensus on the modeling indoor spaces and their neighborhood relationships to support indoor location-based services, a UML Conceptual Model, IndoorGML 2.0 Part 1, was approved as an OGC Standard (OGC 22-045r5) in June, 2025. This model covers geometric and semantic properties of indoor spaces relevant for indoor navigation. This IndoorGML 2.0 Part 2c: SQL Encoding Standard defines how those concepts should be realized using a relational database schema. The SQL encoding of IndoorGML 2.0 is intended for software developers, data providers, standards implementers, and service providers who need a persistent, queryable, and interoperable representation of IndoorGML datasets in SQL databases.

IV SECURITY CONSIDERATIONS

No security considerations have been made for this Standard.

V SUBMITTING ORGANIZATIONS

The following organizations submitted this Document to the Open Geospatial Consortium (OGC):

- University of New South Wales
- SpatialAlgebra Technology Limited
- Pusan National University
- CityGeometrix Pty Ltd
- National Institute of Advanced Industrial Science and Technology

VI SUBMITTERS

All questions regarding this submission should be directed to the editor or the submitters:

NAME	AFFILIATION	EMAIL	OGC MEMBER
Sisi Zlatanova	University of New South Wales	s.zlatanova@unsw.edu.au	Yes
Zhiyong Wang	SpatialAlgebra Technology Limited	zhiyongwang@spatialalgebra.tech	Yes
Taehoon Kim	National Institute of Advanced Industrial Science and Technology	kim.taehoon@aist.go.jp	Yes
Abdoulaye Diakite	CityGeometrix Pty Ltd	abdou@citygeometrix.com	Yes
Ki-Joune Li	Pusan National University	lik@pnu.edu	Yes

1 SCOPE

1 SCOPE

The OGC IndoorGML 2.0 Part 2c – SQL Encoding Standard defines a SQL encoding for IndoorGML 2.0 Part 1 — Conceptual Model.

This Standard specifies:

- SQL table definitions for IndoorGML 2.0 core module;
- SQL table definitions for IndoorGML 2.0 navigation module;
- the geometry storage rules based on PostGIS geometry columns; and
- a statement to which SQL encoding conformance classes an IndoorGML 2.0 implementation conforms to.

The SQL encoding of IndoorGML 2.0 is an implementation encoding of IndoorGML 2.0 Part 1. Therefore, database instances of the SQL encoding shall be interpreted according to the semantics defined in the IndoorGML 2.0 conceptual model.

2 CONFORMANCE

2 CONFORMANCE

This standard defines an implementation specification which specifies how the IndoorGML 2.0 Conceptual Model should be implemented using Structured Query Language (SQL). The Standardization Target for this standard is:

- Implementations of the IndoorGML 2.0 Conceptual Model using SQL encodings.

Conformance with this standard shall be checked using all the relevant tests specified in Annex A (normative) of this document. The framework, concepts, and methodology for testing, and the criteria to be achieved to claim conformance are specified in the OGC Compliance Testing Policies and Procedures and the OGC Compliance Testing web site.

In order to conform to this OGC® interface standard, a software implementation shall choose to implement:

- Any one of the conformance levels specified in Annex A (normative).

All requirements-classes and conformance-classes described in this document are owned by the standard(s) identified.

Table 2

CONFORMANCE CLASS	URI
IndoorGML Core	http://www.opengis.net/spec/indoorgml-2/2.0/sql/conf/core
Navigation Extension	http://www.opengis.net/spec/indoorgml-2/2.0/sql/conf/navi

3

NORMATIVE REFERENCES

3 NORMATIVE REFERENCES

The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

Sisi Zlatanova, Abdoulaye Diakite, Taehoon Kim, Ki-Joune Li, Open Geospatial Consortium (OGC): OGC 22-045r5, *OGC IndoorGML 2.0 Part 1 – Conceptual Model*. Open Geospatial Consortium (2025). http://www.opengis.net/doc/IS/indoorgml/2.0.

Clemens Portele, Panagiotis (Peter) A. Vretanos, Open Geospatial Consortium (OGC): OGC 21-045r1, *OGC Features and Geometries JSON — Part 1: Core*. Open Geospatial Consortium (2026). http://www.opengis.net/doc/IS/json-fg-1/1.0.

International Organization for Standardization (ISO): ISO 19101-1 (90), *Geographic information — Reference model — Part 1: Fundamentals*. International Organization for Standardization, Geneva https://www.iso.org/standard/59164.html.

International Organization for Standardization (ISO): ISO 19107 (90), *Geographic information — Spatial schema*. International Organization for Standardization, Geneva https://www.iso.org/standard/66175.html.

International Organization for Standardization (ISO): ISO/IEC 9075 (95), *Information technology — Database languages — SQL*. International Organization for Standardization; International Electrotechnical Commission, Geneva https://www.iso.org/standard/16663.html.

PostGIS Spatial and Geographic Objects for PostgreSQL, https://postgis.net/docs/

4 TERMS AND DEFINITIONS

4 TERMS AND DEFINITIONS

This document uses the terms defined in OGC Policy Directive 49, which is based on the ISO/IEC Directives, Part 2, Rules for the structure and drafting of International Standards. In particular, the word “shall” (not “must”) is the verb form used to indicate a requirement to be strictly followed to conform to this document and OGC documents do not use the equivalent phrases in the ISO/IEC Directives, Part 2.

This document also uses terms defined in the OGC Standard for Modular specifications (OGC 08-131r3), also known as the ‘ModSpec’. The definitions of terms such as standard, specification, requirement, and conformance test are provided in the ModSpec.

For the purposes of this document, the following additional terms and definitions apply.

4.1. conceptual model

model that defines concepts of a universe of discourse

[**SOURCE:** ISO 19101-1]

4.2. conceptual schema

formal description of a conceptual model

[**SOURCE:** ISO 19101-1]

4.3. feature

abstraction of real world phenomena

Note 1 to entry: Features are objects that have an identity that allows for distinguishing features from each other. Features can have spatial and non-spatial properties that describe the features in more detail. Spatial properties are properties that have a geometry from ISO 19107 as data type.

Note 2 to entry: Features are denoted by the stereotype «FeatureType» in the IndoorGML 2.0 Conceptual Model.

[**SOURCE:** ISO 19101-1]

4.4. relational table

persistent data structure in a relational database that stores records of a single feature type or association

Note 1 to entry: In the SQL encoding of IndoorGML 2.0, each IndoorGML 2.0 feature class is mapped to one or more relational tables.

5 CONVENTIONS

5 CONVENTIONS

5.1. Symbols (and abbreviated terms)

The following symbols and abbreviated terms are used in this standard. Symbols and abbreviated terms defined in IndoorGML 2.0 Part 1 also apply to this document.

Table 3 — Symbols and abbreviated terms

ABBREVIATION	WORD OR PHRASE
DDL	Data Definition Language
EA	Enterprise Architecture
IndoorJSON	JSON encoding of IndoorGML concepts
JSON	JavaScript Object Notation
PostgreSQL	Open-source relational database management system
PostGIS	Spatial database extension for PostgreSQL
SQL	Structured Query Language

5.2. Identifiers

The normative provisions in this standard are denoted by the URI

http://www.opengis.net/spec/indoorgml-2/2.0/sql

All requirements and conformance tests that appear in this document are denoted by partial URIs which are relative to this base.

5.3. Use of SQL terminology

The following terms are used as specified in SQL. In particular:

- “table” refers to a named relation in the database schema;
- “column” refers to an attribute of a table; and
- “row” refers to a single record in a table.

Feature identifiers are stored as string values in varchar columns rather than integer auto-increment keys, in order to preserve cross-references between IndoorGML objects and to align with IndoorJSON identifier conventions.

5.4. Geometry storage

UML geometry DataTypes are flattened onto the owning feature table as PostGIS geometry columns whose names follow the property path (aligned with IndoorJSON member names):

- CellSpace.cellSpaceGeom_geometry2D and CellSpace.cellSpaceGeom_geometry3D (corresponding to CellSpaceGeometryType);
- CellBoundary.cellBoundaryGeom_geometry1D and CellBoundary.cellBoundaryGeom_geometry2D (corresponding to CellBoundaryGeometryType);
- Geometry on Node and Edge.

At most one of the paired geometry columns on CellSpace or CellBoundary is populated for a given row (enforced by a table CHECK constraint). The reference implementation requires the PostGIS extension (PostGIS).

5.5. Complex DataType storage

Non-geometric UML DataTypes that are not feature classes are encoded as PostgreSQL composite types (CREATE TYPE ... AS (...)) rather than as standalone tables. In particular, ExternalObjectReferenceType and ExternalReferenceType are composite types, and CellSpace / CellBoundary store an optional externalReference column of type ExternalReferenceType.

5.6. Code list storage

IndoorGML code list types (for example, ThemeLayerValue, TopoExpressionValue, LocomotionAccessType) are represented as PostgreSQL ENUM types whose labels match the UML type names and enumeration literals defined in the IndoorGML 2.0 XML schema.

5.7. Graph and route associations

Graph adjacency in the dual space layer is stored as jsonb identifier arrays on the owning table:

- Node.connects stores the identifiers of connected Edge rows;
- Edge.connects stores the identifiers of the two connected Node rows.

A Route represents an ordered navigation path through the dual-space graph. The ordered routeNode and routeEdge members are stored as jsonb arrays on the Route table rather than in separate junction tables. This encoding preserves the sequence of nodes and edges in a single row and simplifies maintenance of route order during import, update, and export. Array position expresses traversal order; the first element is the first node or edge in the route, the second element is the next, and so on.

Example:

```
-- routeNode and routeEdge preserve traversal order in array position
INSERT INTO "Route" ("RouteID", "routeNode", "routeEdge")
VALUES (
   'R1',
   '["N1", "N2", "N3"]'::jsonb,
   '["E1", "E2"]'::jsonb
);
```

Listing 1

5.8. Table and column naming

SQL table and column names use identifiers enclosed in double quotes. Table names use PascalCase aligned with UML type names (for example, IndoorFeatures, ThematicLayer, CellSpace, NavigableSpace, ObjectSpace).

6

OVERVIEW OF THE SQL ENCODING

6 OVERVIEW OF THE SQL ENCODING

The SQL encoding of IndoorGML 2.0 implements the conceptual classes of IndoorGML 2.0 Part 1 as relational database tables. The conceptual model remains normative for semantics, while this document defines the SQL representation.

6.1. Design Principle

The SQL encoding is based on the following encoding principles.

6.1.1. Conceptual model preservation

The SQL encoding of the IndoorGML core module preserves the IndoorGML 2.0 core conceptual model structure, as shown in Figure 1.

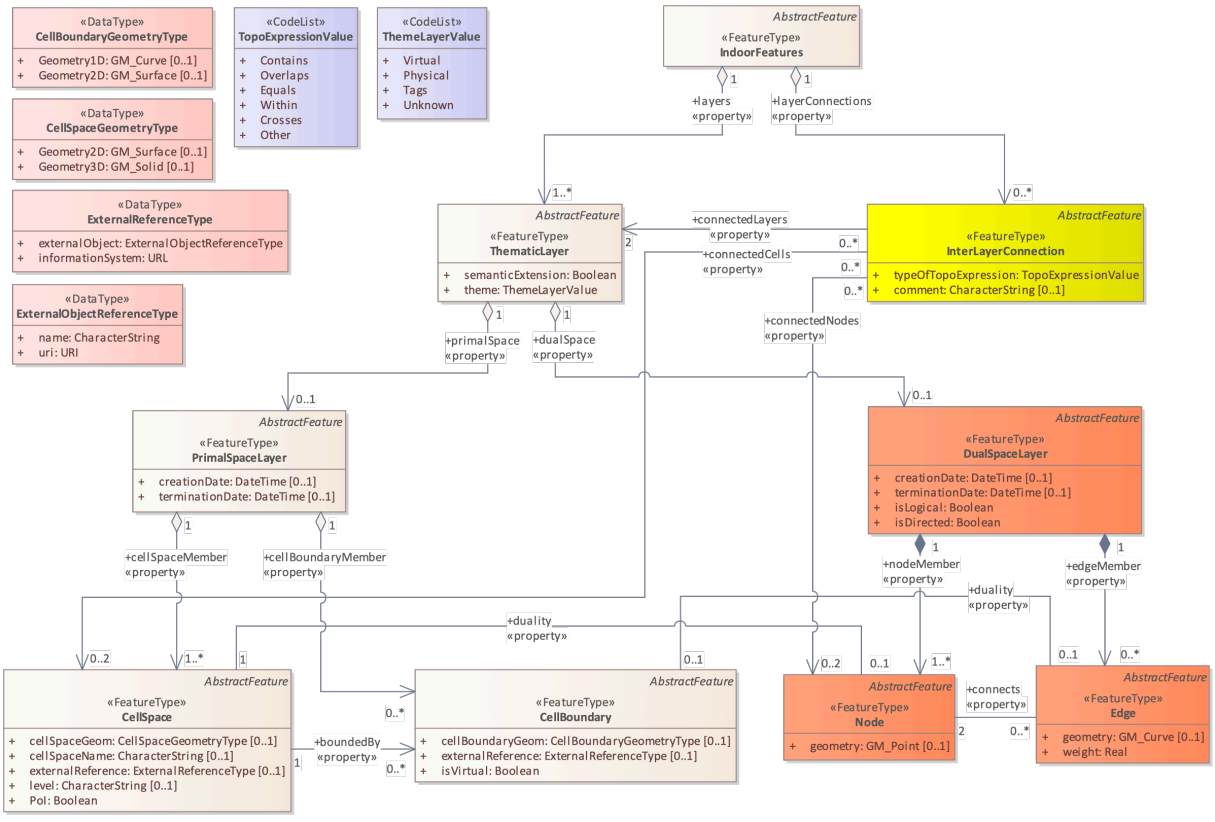


Figure 1 — IndoorGML 2.0 core module structure

Each feature class is mapped to one or more relational tables, and associations are represented using foreign key columns or junction tables. For example, the IndoorFeatures table is related

to ThematicLayer rows through the IndoorFeaturesID foreign key, and InterLayerConnection rows are linked to ThematicLayer, Node, and CellSpace rows through the InterLayerConnection_ThematicLayer, InterLayerConnection_Node, and InterLayerConnection_CellSpace junction tables.

The SQL encoding of the IndoorGML navigation module preserves the IndoorGML 2.0 navigation conceptual model structure, as shown in Figure 2.

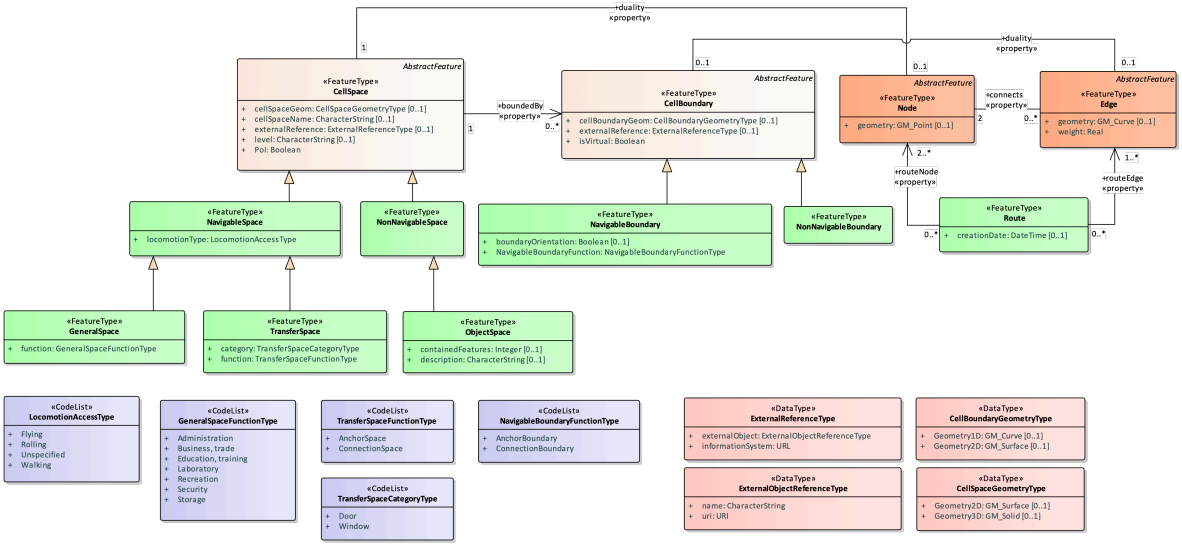


Figure 2 — IndoorGML 2.0 navigation module structure

Navigation feature classes extend core tables using table inheritance patterns implemented through foreign key references to CellSpace and CellBoundary.

6.1.2. UML-to-SQL generation workflow

The SQL encoding schema is derived from the IndoorGML 2.0 UML model and is designed on the basis of PostgreSQL with PostGIS as the reference database platform. SQL CREATE TABLE statements are first generated from the UML model using Enterprise Architecture tooling, and the resulting DDL is then corrected to align with the conceptual model. Typical corrections include converting identifier columns from integer to string (varchar), replacing EA code list tables with PostgreSQL ENUM types named after the UML code list types, flattening geometry DataTypes onto owning feature tables as path-named PostGIS columns (for example, cellSpaceGeom_geometry2D / cellSpaceGeom_geometry3D), replacing EA tables for ExternalReferenceType / ExternalObjectReferenceType with PostgreSQL composite types, storing connects as jsonb arrays on Node and Edge, storing routeNode and routeEdge as ordered jsonb arrays on Route to preserve traversal sequence, adding missing properties (for example, the layers member on IndoorFeatures), and updating relationship cardinalities and junction tables.

7

SQL ENCODING OF THE CORE MODULE

7 SQL ENCODING OF THE CORE MODULE

REQUIREMENTS CLASS 1	
IDENTIFIER	http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
TARGET TYPE	Implementation Specification
PREREQUISITES	http://www.opengis.net/spec/indoorgml/2.0/req http://www.opengis.net/spec/json-fg-1/1.0/req/core http://www.opengis.net/spec/json-fg-1/1.0/req/polyhedra
NORMATIVE STATEMENTS	Requirement 1: /req/core/indoorsql Requirement 2: /req/core/indoorfeature Requirement 3: /req/core/thematiclayer Requirement 4: /req/core/primalspacelayer Requirement 5: /req/core/cellspace Requirement 6: /req/core/cellboundary Requirement 7: /req/core/dualspacelayer Requirement 8: /req/core/node Requirement 9: /req/core/edge Requirement 10: /req/core/interlayerconnection

Each feature table of the SQL encoding represents one feature class of the IndoorGML 2.0 conceptual model.

Every feature table:

- **must** have a primary key column storing a string identifier that serves as a unique identifier for the feature and can be used to represent cross-references between objects in the SQL encoding.

REQUIREMENT 1: REQUIREMENT FOR SQL ENCODING FEATURE TABLES	
IDENTIFIER	/req/core/indoorsql
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	Each feature table of the SQL encoding SHALL contain a string primary key column for the feature identifier.

7.1. Core code list types

The following PostgreSQL ENUM types represent IndoorGML core code lists:

- ThemeLayerValue for the ThematicLayer.Theme column;
- TopoExpressionValue for the InterLayerConnection.Typeoftopoexpression column.

```
CREATE TYPE "ThemeLayerValue" AS ENUM (
   'Virtual',
   'Physical',
   'Tags',
   'Unknown'
);

CREATE TYPE "TopoExpressionValue" AS ENUM (
   'Contains',
   'Overlaps',
   'Equals',
   'Within',
   'Crosses',
   'Other'
);
```

Listing 2 — A SQL schema of the core code list types:

7.2. IndoorFeatures

The IndoorFeatures table is an instance of IndoorFeature, the container for the indoor dataset. It aggregates one or more thematic layers and may include inter-layer connections.

The IndoorFeatures table shall contain:

- IndoorFeaturesID as the primary key
- optionally, layers as a reference to associated thematic layers

NOTE:The layers column was added during schema correction because it was not included in the SQL generated from Enterprise Architecture tooling.

```
CREATE TABLE "IndoorFeatures"
(
   "IndoorFeaturesID" varchar(100) NOT NULL,
   "layers" varchar(100) NULL,
   CONSTRAINT "PK_IndoorFeatures" PRIMARY KEY ("IndoorFeaturesID")
);
```

Listing 3 — A SQL schema of the IndoorFeatures table:

```
INSERT INTO "IndoorFeatures" ("IndoorFeaturesID", "layers")
```

```
VALUES ('IF-1', 'TL-1');

-- Additional thematic layers and inter-layer connections may be inserted here.
```

Listing 4 — An example INSERT statement for the IndoorFeatures table:

REQUIREMENT 2: INDOORFEATURES SCHEMA	
IDENTIFIER	/req/core/indoorfeature
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	An IndoorFeatures table SHALL be related to at least one ThematicLayer row through a foreign key.

7.3. ThematicLayer

The ThematicLayer table is an instance of ThematicLayer, which represents one contextual decomposition of indoor space.

The ThematicLayer table shall contain:

- ThematicLayerID as the primary key
- IndoorFeaturesID as a foreign key to IndoorFeatures
- Semanticextension indicates whether there is an extension module with additional semantic information
- Theme of type ThemeLayerValue, with values including “Virtual”, “Physical”, “Tags”, and “Unknown”

```
CREATE TABLE "ThematicLayer"
(
   "Semanticextension" boolean NULL,
   "Theme" "ThemeLayerValue" NULL,
   "ThematicLayerID" varchar(100) NOT NULL,
   "IndoorFeaturesID" varchar(100) NOT NULL,
   CONSTRAINT "PK_ThematicLayer" PRIMARY KEY ("ThematicLayerID"),
   CONSTRAINT "FK_ThematicLayer_layers"
      FOREIGN KEY ("IndoorFeaturesID") REFERENCES "IndoorFeatures" ("IndoorFeaturesID")
);

COMMENT ON TABLE "ThematicLayer"
   IS 'A layer of specific theme aggregating a primal and/or a dual space of a given environment.';

COMMENT ON COLUMN "ThematicLayer"."Semanticextension"
   IS 'Indicates whether semantic information is associated (true) or not (false) to the primal space of
the thematic layer.';
```

```
COMMENT ON COLUMN "ThematicLayer"."Theme"
  IS 'Determines the theme of the layer (e.g topographic, logical, etc.).';
```

Listing 5 — A SQL schema of the ThematicLayer table:

REQUIREMENT 3: THEMATICLAYER SCHEMA	
IDENTIFIER	/req/core/thematiclayer
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	A ThematicLayer table SHALL declare whether there is an extension module with additional semantic information using the Semanticextension column.
B	A ThematicLayer table SHALL determine what type of model representation can be expected using the Theme column.
C	The Theme value SHALL be one of “Virtual”, “Physical”, “Tags”, and “Unknown”.

7.4. PrimalSpaceLayer

The PrimalSpaceLayer table is an instance of PrimalSpaceLayer, which contains spatial objects representing indoor subdivisions.

The PrimalSpaceLayer table shall contain:

- PrimalSpaceLayerID as the primary key
- ThematicLayerID as a foreign key to ThematicLayer
- optionally, Creationdate
- optionally, Terminationdate

```
CREATE TABLE "PrimalSpaceLayer"
(
   "Creationdate" timestamp without time zone NULL,
   "Terminationdate" timestamp without time zone NULL,
   "PrimalSpaceLayerID" varchar(100) NOT NULL,
   "ThematicLayerID" varchar(100) NULL,
   CONSTRAINT "PK_PrimalSpaceLayer" PRIMARY KEY ("PrimalSpaceLayerID"),
   CONSTRAINT "FK_PrimalSpaceLayer_primalSpace"
      FOREIGN KEY ("ThematicLayerID") REFERENCES "ThematicLayer" ("ThematicLayerID")
);
```

Listing 6 — A SQL schema of the PrimalSpaceLayer table:

REQUIREMENT 4: PRIMALSPACELAYER SCHEMA	
IDENTIFIER	/req/core/primalspacelayer
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	A PrimalSpaceLayer table SHALL be related to one or more CellSpace rows.

7.5. CellSpace

The CellSpace table is an instance of CellSpace, which represents a unit of indoor space.

The CellSpace table shall contain:

- CellSpaceID as the primary key
- PrimalSpaceLayerID as a foreign key to PrimalSpaceLayer
- Poi is a flag that specifies whether this cell space is a point of interest
- optionally, CellSpaceName
- optionally, Level
- optionally, duality referencing the NodeID of the dual node
- optionally, cellSpaceGeom_geometry2D as a PostGIS geometry value (UML CellSpaceGeometryType.Geometry2D / IndoorJSON cellSpaceGeom.geometry2D)
- optionally, cellSpaceGeom_geometry3D as a PostGIS geometry value (UML CellSpaceGeometryType.Geometry3D / IndoorJSON cellSpaceGeom.geometry3D)
- optionally, externalReference of composite type ExternalReferenceType

At most one of cellSpaceGeom_geometry2D and cellSpaceGeom_geometry3D is populated for a given row.

CellBoundary rows reference CellSpace rows through the bounds foreign key column.

```
CREATE TABLE "CellSpace"
(
   "cellSpaceGeom_geometry2D" geometry NULL,
   "cellSpaceGeom_geometry3D" geometry NULL,
   "CellSpaceName" varchar(100) NULL,
   "externalReference" "ExternalReferenceType" NULL,
   "Level" varchar(100) NULL,
   "Poi" boolean NULL,
```

```
    "CellSpaceID" varchar(100) NOT NULL,
    "PrimalSpaceLayerID" varchar(100) NOT NULL,
    "duality" varchar(100) NULL,
    CONSTRAINT "PK_CellSpace" PRIMARY KEY ("CellSpaceID"),
    CONSTRAINT "chk_CellSpace_geom_xor" CHECK (
      NOT ("cellSpaceGeom_geometry2D" IS NOT NULL AND "cellSpaceGeom_geometry3D" IS NOT NULL)
    ),
    CONSTRAINT "FK_CellSpace_cellSpaceMember"
      FOREIGN KEY ("PrimalSpaceLayerID") REFERENCES "PrimalSpaceLayer" ("PrimalSpaceLayerID")
);
```

Listing 7 — A SQL schema of the CellSpace table:

```
SELECT
    "CellSpaceID"              AS id,
    'CellSpace'                AS "featureType",
    "Poi"                      AS poi,
    "cellSpaceGeom_geometry2D"   AS "geometry2D",
    "cellSpaceGeom_geometry3D"   AS "geometry3D",
    "CellSpaceName"            AS name,
    "duality"                  AS duality,
    "Level"                    AS level,
    "externalReference"        AS "externalReference"
FROM "CellSpace"
ORDER BY "CellSpaceID";
```

Listing 8 — An example query returning CellSpace records:

REQUIREMENT 5: CELLSPACE SCHEMA	
IDENTIFIER	/req/core/cellspace
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	A CellSpace table SHALL declare whether it is a point of interest space using the Poi column.

7.6. CellBoundary

The CellBoundary table is an instance of CellBoundary, a boundary of a CellSpace.

The CellBoundary table shall contain:

- CellBoundaryID as the primary key
- PrimalSpaceLayerID as a foreign key to PrimalSpaceLayer
- Isvirtual indicating whether the boundary is virtual or physical
- optionally, bounds referencing the CellSpaceID of a bounded cell space

- optionally, cellBoundaryGeom_geometry1D as a PostGIS geometry value (UML CellBoundaryGeometryType.Geometry1D, typically a curve)
- optionally, cellBoundaryGeom_geometry2D as a PostGIS geometry value (UML CellBoundaryGeometryType.Geometry2D, typically a surface)
- optionally, externalReference of composite type ExternalReferenceType

At most one of cellBoundaryGeom_geometry1D and cellBoundaryGeom_geometry2D is populated for a given row. NOTE: IndoorJSON names the curve member cellBoundaryGeom.geometry2D and the surface member cellBoundaryGeom.geometry3D; the SQL columns follow the UML Geometry1D / Geometry2D names.

```
CREATE TABLE "CellBoundary"
(
   "cellBoundaryGeom_geometry1D" geometry NULL,
   "cellBoundaryGeom_geometry2D" geometry NULL,
   "externalReference" "ExternalReferenceType" NULL,
   "Isvirtual" boolean NULL,
   "CellBoundaryID" varchar(100) NOT NULL,
   bounds varchar(100) NULL,
   "PrimalSpaceLayerID" varchar(100) NULL,
   CONSTRAINT "PK_CellBoundary" PRIMARY KEY ("CellBoundaryID"),
   CONSTRAINT "chk_CellBoundary_geom_xor" CHECK (
      NOT ("cellBoundaryGeom_geometry1D" IS NOT NULL AND "cellBoundaryGeom_geometry2D" IS
NOT NULL)
   ),
   CONSTRAINT "FK_CellBoundary_boundedBy"
      FOREIGN KEY (bounds) REFERENCES "CellSpace" ("CellSpaceID"),
   CONSTRAINT "FK_CellBoundary_cellBoundaryMember"
      FOREIGN KEY ("PrimalSpaceLayerID") REFERENCES "PrimalSpaceLayer" ("PrimalSpaceLayerID")
);
```

Listing 9 — A SQL schema of the CellBoundary table:

REQUIREMENT 6: CELLBOUNDARY SCHEMA	
IDENTIFIER	/req/core/cellboundary
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	A CellBoundary table SHALL specify whether it is virtual using the Isvirtual column.

7.7. DualSpaceLayer

The DualSpaceLayer table is an instance of DualSpaceLayer, representing the graph structure corresponding to the primal space layer.

The DualSpaceLayer table shall contain:

- DualSpaceLayerID as the primary key
- ThematicLayerID as a foreign key to ThematicLayer
- Islogical indicating whether the graph is geometric or logical
- Isdirected specifying whether the graph is directed
- optionally, Creationdate
- optionally, Terminationdate

```
CREATE TABLE "DualSpaceLayer"
(
   "Creationdate" timestamp without time zone NULL,
   "Terminationdate" timestamp without time zone NULL,
   "Islogical" boolean NULL,
   "Isdirected" boolean NULL,
   "DualSpaceLayerID" varchar(100) NOT NULL,
   "ThematicLayerID" varchar(100) NULL,
   CONSTRAINT "PK_DualSpaceLayer" PRIMARY KEY ("DualSpaceLayerID"),
   CONSTRAINT "FK_DualSpaceLayer_dualSpace"
      FOREIGN KEY ("ThematicLayerID") REFERENCES "ThematicLayer" ("ThematicLayerID")
);
```

Listing 10 — A SQL schema of the DualSpaceLayer table:

REQUIREMENT 7: DUALSPACELAYER SCHEMA	
IDENTIFIER	/req/core/dualspacelayer
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	A DualSpaceLayer table SHALL specify whether the provided graph is geometric or logical using the Islogical column.
B	A DualSpaceLayer table SHALL specify whether the provided graph is directed using the Isdirected column.
C	A DualSpaceLayer table SHALL be related to one or more Node rows.

7.8. Node

The Node table is an instance of Node, a state or graph vertex in the dual space layer.

The Node table shall contain:

- NodeID as the primary key
- DualSpaceLayerID as a foreign key to DualSpaceLayer
- duality referencing the CellSpaceID of the corresponding cell space
- optionally, Geometry as a PostGIS geometry value (typically Point)
- optionally, connects as a jsonb array of connected EdgeID values

```
CREATE TABLE "Node"
(
   "Geometry" geometry NULL,
   "NodeID" varchar(100) NOT NULL,
   "duality" varchar(100) NULL,
   "DualSpaceLayerID" varchar(100) NOT NULL,
   connects jsonb NULL,
   CONSTRAINT "PK_Node" PRIMARY KEY ("NodeID"),
   CONSTRAINT "FK_Node_duality"
      FOREIGN KEY ("duality") REFERENCES "CellSpace" ("CellSpaceID"),
   CONSTRAINT "FK_Node_DualSpaceLayer"
      FOREIGN KEY ("DualSpaceLayerID") REFERENCES "DualSpaceLayer" ("DualSpaceLayerID")
);

ALTER TABLE "CellSpace" ADD CONSTRAINT "FK_CellSpace_duality"
   FOREIGN KEY ("duality") REFERENCES "Node" ("NodeID");
```

Listing 11 — A SQL schema of the Node table:

```
SELECT
   n."NodeID" AS id,
   'Node' AS "featureType",
   n."Geometry" AS geometry,
   n."duality" AS duality,
   n.connects AS connects
FROM "Node" n
ORDER BY n."NodeID";
```

Listing 12 — An example query returning Node records:

REQUIREMENT 8: NODE SCHEMA	
IDENTIFIER	/req/core/node
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	A Node table SHALL contain a duality value referencing the CellSpaceID of the corresponding Cell Space.
B	A Node table SHALL store connected edge identifiers in the connects column.

7.9. Edge

The Edge table is an instance of Edge, representing a relationship between two nodes.

The Edge table shall contain:

- EdgeID as the primary key
- DualSpaceLayerID as a foreign key to DualSpaceLayer
- Weight for use in graph-based applications
- optionally, duality referencing the CellBoundaryID of the corresponding cell boundary
- optionally, Geometry as a PostGIS geometry value (typically LineString)
- optionally, connects as a jsonb array of the two connected NodeID values

```
CREATE TABLE "Edge"
(
   "Geometry" geometry NULL,
   "Weight" real NULL,
   "EdgeID" varchar(100) NOT NULL,
   "duality" varchar(100) NULL,
   "DualSpaceLayerID" varchar(100) NULL,
   connects jsonb NULL,
   CONSTRAINT "PK_Edge" PRIMARY KEY ("EdgeID"),
   CONSTRAINT "FK_Edge_duality"
      FOREIGN KEY ("duality") REFERENCES "CellBoundary" ("CellBoundaryID"),
   CONSTRAINT "FK_Edge_DualSpaceLayer"
      FOREIGN KEY ("DualSpaceLayerID") REFERENCES "DualSpaceLayer" ("DualSpaceLayerID")
);
```

Listing 13 — A SQL schema of the Edge table:

REQUIREMENT 9: EDGE SCHEMA	
IDENTIFIER	/req/core/edge
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	An Edge table SHALL reference connected Nodes using the connects column.
B	An Edge table SHALL contain a Weight column.

7.10. InterLayerConnection

The InterLayerConnection table is an instance of InterLayerConnection, representing a relationship between objects in different thematic layers.

The InterLayerConnection table shall contain:

- InterLayerConnectionID as the primary key
- IndoorFeaturesID as a foreign key to IndoorFeatures
- Typeoftopoexpression of type TopoExpressionValue, with values including “Contains”, “Overlaps”, “Equals”, “Within”, “Crosses”, and “Other”
- optionally, Comment

Associated junction tables InterLayerConnection_ThematicLayer, InterLayerConnection_Node, and InterLayerConnection_CellSpace store references to thematic layers, nodes, and cell spaces respectively.

```
CREATE TABLE "InterLayerConnection"
(
   "Typeoftopoexpression" "TopoExpressionValue" NULL,
   "Comment" text NULL,
   "InterLayerConnectionID" varchar(100) NOT NULL,
   "IndoorFeaturesID" varchar(100) NULL,
   CONSTRAINT "PK_InterLayerConnection" PRIMARY KEY ("InterLayerConnectionID"),
   CONSTRAINT "FK_InterLayerConnection_layerConnections"
      FOREIGN KEY ("IndoorFeaturesID") REFERENCES "IndoorFeatures" ("IndoorFeaturesID")
);

COMMENT ON TABLE "InterLayerConnection"
   IS 'Describes the connection between two thematic layers. ';

COMMENT ON COLUMN "InterLayerConnection"."Typeoftopoexpression"
   IS 'Describes the topological relationship between two layers (e.g. overlaps, contains, etc.).';
```

Listing 14 — A SQL schema of the InterLayerConnection table:

```
CREATE TABLE "InterLayerConnection_CellSpace"
(
   "connectedCells" varchar(100) NULL,
   "InterLayerConnectionID" varchar(100) NULL,
   CONSTRAINT "FK_InterLayerConnection_CellSpace_connectedCells"
      FOREIGN KEY ("connectedCells") REFERENCES "CellSpace" ("CellSpaceID"),
   CONSTRAINT "FK_InterLayerConnection_CellSpace_InterLayerConnection"
      FOREIGN KEY ("InterLayerConnectionID") REFERENCES "InterLayerConnection"
("InterLayerConnectionID")
);
```

Listing 15 — A SQL schema of the InterLayerConnection_CellSpace junction table:

```
CREATE TABLE "InterLayerConnection_Node"
(
   "connectedNodes" varchar(100) NULL,
   "InterLayerConnectionID" varchar(100) NULL,
```

```
    CONSTRAINT "FK_InterLayerConnection_Node_connectedNodes"
      FOREIGN KEY ("connectedNodes") REFERENCES "Node" ("NodeID"),
    CONSTRAINT "FK_InterLayerConnection_Node_InterLayerConnection"
      FOREIGN KEY ("InterLayerConnectionID") REFERENCES "InterLayerConnection"
("InterLayerConnectionID")
);
```

Listing 16 — A SQL schema of the InterLayerConnection_Node junction table:

```
CREATE TABLE "InterLayerConnection_ThematicLayer"
(
    "connectedLayers" varchar(100) NULL,
    "InterLayerConnectionID" varchar(100) NULL,
    CONSTRAINT "FK_InterLayerConnection_ThematicLayer_connectedLayers"
      FOREIGN KEY ("connectedLayers") REFERENCES "ThematicLayer" ("ThematicLayerID"),
    CONSTRAINT "FK_InterLayerConnection_ThematicLayer_InterLayerConnection"
      FOREIGN KEY ("InterLayerConnectionID") REFERENCES "InterLayerConnection"
("InterLayerConnectionID")
);
```

Listing 17 — A SQL schema of the InterLayerConnection_ThematicLayer junction table:

REQUIREMENT 10: INTERLAYERCONNECTION SCHEMA	
IDENTIFIER	/req/core/interlayerconnection
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	An InterLayerConnection table SHALL reference ThematicLayer rows through the InterLayerConnection_ThematicLayer junction table.

7.11. ExternalReferenceType

The UML DataTypes ExternalObjectReferenceType and ExternalReferenceType are encoded as PostgreSQL composite types rather than as tables. These are not feature classes of IndoorGML 2.0. CellSpace and CellBoundary optionally store an externalReference column of type ExternalReferenceType, which nests an ExternalObject of type ExternalObjectReferenceType together with an InformationSystem URI.

```
/* UML DataType ExternalObjectReferenceType / ExternalReferenceType
   Encoded as PostgreSQL composite types (not tables).
   Apply before creating CellSpace / CellBoundary. */

CREATE TYPE "ExternalObjectReferenceType" AS (
    "Name" varchar(100),
    "Uri" text
);

CREATE TYPE "ExternalReferenceType" AS (
    "ExternalObject" "ExternalObjectReferenceType",
    "InformationSystem" text
```

);
```

**Listing 18 — A SQL schema of the ExternalReference composite types:**
```

8

SQL ENCODING OF THE NAVIGATION MODULE

8 SQL ENCODING OF THE NAVIGATION MODULE

REQUIREMENTS CLASS 2	
IDENTIFIER	http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/navi
TARGET TYPE	Implementation Specification
PREREQUISITES	http://www.opengis.net/spec/indoorgml/2.0/req http://www.opengis.net/spec/json-fg-1/1.0/req/core http://www.opengis.net/spec/json-fg-1/1.0/req/polyhedra
NORMATIVE STATEMENTS	Requirement 11: /req/navi/navigablespace Requirement 12: /req/navi/generalspace Requirement 13: /req/navi/transferspace Requirement 14: /req/navi/objectspace Requirement 15: /req/navi/navigableboundary Requirement 16: /req/navi/route

The navigation SQL encoding schema extends the core SQL encoding schema with tables for navigation-related feature classes.

NOTE: Navigation feature classes have not yet been validated with actual datasets in the preliminary implementation.

8.1. Navigation code list types

The following PostgreSQL ENUM types represent IndoorGML navigation code lists:

- LocomotionAccessType for NavigableSpace.Locomotiontype;
- GeneralSpaceFunctionType for GeneralSpace.Function;
- TransferSpaceFunctionType for TransferSpace.Function;
- TransferSpaceCategoryType for TransferSpace.Type;
- NavigableBoundaryFunctionType for NavigableBoundary.NavigableBoundaryFunction.

```
CREATE TYPE "GeneralSpaceFunctionType" AS ENUM (
   'Administration',
```

```
    'Business, trade',
    'Education, training',
    'Recreation',
    'Laboratory',
    'Storage',
    'Security'
);

CREATE TYPE "LocomotionAccessType" AS ENUM (
    'Walking',
    'Flying',
    'Rolling',
    'Unspecified'
);

CREATE TYPE "NavigableBoundaryFunctionType" AS ENUM (
    'AnchorBoundary',
    'ConnectionBoundary'
);

CREATE TYPE "TransferSpaceFunctionType" AS ENUM (
    'ConnectionSpace',
    'AnchorSpace'
);

CREATE TYPE "TransferSpaceCategoryType" AS ENUM (
    'Door',
    'Window'
);
```

Listing 19 — A SQL schema of the navigation code list types:

8.2. NavigableSpace

The NavigableSpace table extends CellSpace with navigation-specific attributes.

The NavigableSpace table shall contain:

- NavigableSpaceID as a primary key and foreign key to CellSpace
- Locomotiontype of type LocomotionAccessType, with values including “Walking”, “Flying”, “Rolling”, and “Unspecified”

```
CREATE TABLE "NavigableSpace"
(
    "Locomotiontype" "LocomotionAccessType" NULL,
    "NavigableSpaceID" varchar(100) NOT NULL,
    CONSTRAINT "PK_NavigableSpace" PRIMARY KEY ("NavigableSpaceID"),
    CONSTRAINT "FK_NavigableSpace_CellSpace"
        FOREIGN KEY ("NavigableSpaceID") REFERENCES "CellSpace" ("CellSpaceID")
);
```

Listing 20 — A SQL schema of the NavigableSpace table:

REQUIREMENT 11: NAVIGABLESPACE SCHEMA	
IDENTIFIER	/req/navi/navigablespace
INCLUDED IN	Requirements class 2: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/navi
A	A NavigableSpace table SHALL reference a CellSpace row and SHALL contain a Locomotiontype column.

8.3. GeneralSpace

The GeneralSpace table extends NavigableSpace with a function attribute.

The GeneralSpace table shall contain:

- GeneralSpaceID as a primary key and foreign key to NavigableSpace
- Function of type GeneralSpaceFunctionType, with values including “Administration”, “Business, trade”, “Education, training”, “Recreation”, “Laboratory”, “Storage”, and “Security”

```
CREATE TABLE "GeneralSpace"
(
   "Function" "GeneralSpaceFunctionType" NULL,
   "GeneralSpaceID" varchar(100) NOT NULL,
   CONSTRAINT "PK_GeneralSpace" PRIMARY KEY ("GeneralSpaceID"),
   CONSTRAINT "FK_GeneralSpace_NavigableSpace"
      FOREIGN KEY ("GeneralSpaceID") REFERENCES "NavigableSpace" ("NavigableSpaceID")
);
```

Listing 21 — A SQL schema of the GeneralSpace table:

REQUIREMENT 12: GENERALSPACE SCHEMA	
IDENTIFIER	/req/navi/generalspace
INCLUDED IN	Requirements class 2: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/navi
A	A GeneralSpace table SHALL contain a Function column.

8.4. TransferSpace

The TransferSpace table extends NavigableSpace with transfer-specific attributes.

The TransferSpace table shall contain:

- TransferSpaceID as a primary key and foreign key to NavigableSpace
- Type of type TransferSpaceCategoryType, with values “Door” and “Window”
- Function of type TransferSpaceFunctionType, with values “AnchorSpace” and “ConnectionSpace”

```
CREATE TABLE "TransferSpace"
(
   "Function" "TransferSpaceFunctionType" NULL,
   "Type" "TransferSpaceCategoryType" NULL,
   "TransferSpaceID" varchar(100) NOT NULL,
   CONSTRAINT "PK_TransferSpace" PRIMARY KEY ("TransferSpaceID"),
   CONSTRAINT "FK_TransferSpace_NavigableSpace"
      FOREIGN KEY ("TransferSpaceID") REFERENCES "NavigableSpace" ("NavigableSpaceID")
);
```

Listing 22 — A SQL schema of the TransferSpace table:

REQUIREMENT 13: TRANSFERSPACE SCHEMA	
IDENTIFIER	/req/navi/transferspace
INCLUDED IN	Requirements class 2: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/navi
A	A TransferSpace table SHALL contain Type and Function columns.

8.5. NonNavigableSpace

The NonNavigableSpace table extends CellSpace for spaces that are not navigable.

```
CREATE TABLE "NonNavigableSpace"
(
   "NonNavigableSpaceID" varchar(100) NOT NULL,
   CONSTRAINT "PK_NonNavigableSpace" PRIMARY KEY ("NonNavigableSpaceID"),
   CONSTRAINT "FK_NonNavigableSpace_CellSpace"
      FOREIGN KEY ("NonNavigableSpaceID") REFERENCES "CellSpace" ("CellSpaceID")
```

```
);
```

Listing 23 — A SQL schema of the NonNavigableSpace table:

8.6. ObjectSpace

The ObjectSpace table extends NonNavigableSpace with object-specific attributes.

The ObjectSpace table shall contain:

- ObjectSpaceID as a primary key and foreign key to NonNavigableSpace
- optionally, Containedfeatures
- optionally, Description

```
CREATE TABLE "ObjectSpace"
(
   "Description" text NULL,
   "Containedfeatures" varchar(100) NULL,
   "ObjectSpaceID" varchar(100) NOT NULL,
   CONSTRAINT "PK_ObjectSpace" PRIMARY KEY ("ObjectSpaceID"),
   CONSTRAINT "FK_ObjectSpace_NonNavigableSpace"
      FOREIGN KEY ("ObjectSpaceID") REFERENCES "NonNavigableSpace" ("NonNavigableSpaceID")
);
```

Listing 24 — A SQL schema of the ObjectSpace table:

REQUIREMENT 14: OBJECTSPACE SCHEMA	
IDENTIFIER	/req/navi/objectspace
INCLUDED IN	Requirements class 2: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/navi
A	An ObjectSpace table SHALL reference a NonNavigableSpace row.

8.7. NonNavigableBoundary

The NonNavigableBoundary table extends CellBoundary for boundaries that are not navigable.

```
CREATE TABLE "NonNavigableBoundary"
(
   "NonNavigableBoundaryID" varchar(100) NOT NULL,
```

```
    CONSTRAINT "PK_NonNavigableBoundary" PRIMARY KEY ("NonNavigableBoundaryID"),
    CONSTRAINT "FK_NonNavigableBoundary_CellBoundary"
        FOREIGN KEY ("NonNavigableBoundaryID") REFERENCES "CellBoundary" ("CellBoundaryID")
);
```

Listing 25 — A SQL schema of the NonNavigableBoundary table:

8.8. NavigableBoundary

The NavigableBoundary table extends CellBoundary with navigation-specific boundary attributes.

The NavigableBoundary table shall contain:

- NavigableBoundaryID as a primary key and foreign key to CellBoundary
- optionally, Boundaryorientation
- NavigableBoundaryFunction of type NavigableBoundaryFunctionType, with values "AnchorBoundary" and "ConnectionBoundary"

```
CREATE TABLE "NavigableBoundary"
(
    "NavigableBoundaryFunction" "NavigableBoundaryFunctionType" NULL,
    "Boundaryorientation" boolean NULL,
    "NavigableBoundaryID" varchar(100) NOT NULL,
    CONSTRAINT "PK_NavigableBoundary" PRIMARY KEY ("NavigableBoundaryID"),
    CONSTRAINT "FK_NavigableBoundary_CellBoundary"
        FOREIGN KEY ("NavigableBoundaryID") REFERENCES "CellBoundary" ("CellBoundaryID")
);
```

Listing 26 — A SQL schema of the NavigableBoundary table:

REQUIREMENT 15: NAVIGABLEBOUNDARY SCHEMA	
IDENTIFIER	/req/navi/navigableboundary
INCLUDED IN	Requirements class 2: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/navi
A	A NavigableBoundary table SHALL contain a NavigableBoundaryFunction column.

8.9. Route

The Route table represents a navigation route as an ordered sequence of nodes and edges.

In this SQL encoding, routeNode and routeEdge are stored as jsonb arrays on the Route row. This design keeps the full traversal order in one place and makes it easier to maintain, reorder, or replace the node and edge sequence of a route without managing separate ordered junction-table rows. The array index corresponds to position in the route: routeNode[0] is the first node, routeNode[1] is the second node, and routeEdge follows the same rule for edges.

The Route table shall contain:

- RouteID as the primary key
- optionally, Creationdate
- optionally, routeNode as a jsonb array of ordered NodeID values
- optionally, routeEdge as a jsonb array of ordered EdgeID values

```
CREATE TABLE "Route"
(
   "Creationdate" timestamp without time zone NULL,
   "RouteID" varchar(100) NOT NULL,
   "routeNode" jsonb NULL,
   "routeEdge" jsonb NULL,
   CONSTRAINT "PK_Route" PRIMARY KEY ("RouteID")
);
```

Listing 27 — A SQL schema of the Route table:

REQUIREMENT 16: ROUTE SCHEMA	
IDENTIFIER	/req/navi/route
INCLUDED IN	Requirements class 2: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/navi
A	A Route table SHALL store ordered node identifiers in the routeNode column.
B	A Route table SHALL store ordered edge identifiers in the routeEdge column.

A

ANNEX A (NORMATIVE) CONFORMANCE CLASS ABSTRACT TEST SUITE

ANNEX A (NORMATIVE) CONFORMANCE CLASS ABSTRACT TEST SUITE

NOTE: Ensure that there is a conformance class for each requirements class and a test for each requirement (identified by requirement name and number)

A.1. Conformance Class Core

CONFORMANCE TEST A.1	
NORMATIVE STATEMENTS	Recommendation A.1-1 Verify that the database schema implements all core SQL encoding tables and constraints. Requirement A.1-1 conf/indoorsql/core/schema Requirement A.1-2 Verify that all core tables defined in Clause 7 exist with the required columns, primary keys, and foreign key constraints. Verify that identifier columns use string types. Verify that geometry columns use the PostGIS geometry type, including path-named columns such as cellSpaceGeom_geometry2D / cellSpaceGeom_geometry3D. Verify that ExternalReferenceType and ExternalObjectReferenceType exist as PostgreSQL composite types and that externalReference columns use ExternalReferenceType.

A.2. Conformance Class Navigation

CONFORMANCE TEST A.2	
NORMATIVE STATEMENTS	Recommendation A.2-1 Verify that the database schema implements all navigation SQL encoding tables and constraints. Requirement A.2-1 conf/indoorsql/navi/schema Requirement A.2-2 Verify that all navigation tables defined in Clause 8 exist with the required columns and foreign key references to core tables.

B

ANNEX B (NORMATIVE) SCHEMAS

B ANNEX B (NORMATIVE) SCHEMAS

B.1. SQL Schema of IndoorGML Core

```
/* ---------------------------------------------------- */
/*  Generated by Enterprise Architect Version 14.1   */
/*  Created On : 07-7月-2025 11:10:23     */
/*  DBMS       : PostgreSQL       */
/* ---------------------------------------------------- */

CREATE EXTENSION IF NOT EXISTS postgis
;
/* Drop Tables */

DROP TABLE IF EXISTS "CellBoundary" CASCADE
;

DROP TABLE IF EXISTS "CellBoundaryGeometryType" CASCADE
;

DROP TABLE IF EXISTS "CellSpace" CASCADE
;

DROP TABLE IF EXISTS "CellSpaceGeometryType" CASCADE
;

DROP TABLE IF EXISTS "DualSpaceLayer" CASCADE
;

DROP TABLE IF EXISTS "Edge" CASCADE
;

DROP TABLE IF EXISTS "ExternalObjectReferenceType" CASCADE
;

DROP TABLE IF EXISTS "ExternalReferenceType" CASCADE
;

DROP TABLE IF EXISTS "IndoorFeatures" CASCADE
;

DROP TABLE IF EXISTS "InterLayerConnection" CASCADE
;

DROP TABLE IF EXISTS "InterLayerConnection_CellSpace" CASCADE
;

DROP TABLE IF EXISTS "InterLayerConnection_Node" CASCADE
```

```
;

DROP TABLE IF EXISTS "InterLayerConnection_ThematicLayer" CASCADE
;

DROP TABLE IF EXISTS "InterLayerConnection CellSpace" CASCADE
;

DROP TABLE IF EXISTS "InterLayerConnection Node" CASCADE
;

DROP TABLE IF EXISTS "InterLayerConnection ThematicLayer" CASCADE
;

DROP TABLE IF EXISTS "Node" CASCADE
;

DROP TABLE IF EXISTS "PrimalSpaceLayer" CASCADE
;

DROP TABLE IF EXISTS "ThematicLayer" CASCADE
;

DROP TABLE IF EXISTS "ThemeLayerValue" CASCADE
;

DROP TABLE IF EXISTS "TopoExpressionValue" CASCADE
;

/* Drop ENUM and composite Types */

DROP TYPE IF EXISTS "TopoExpressionValue" CASCADE
;

DROP TYPE IF EXISTS "ThemeLayerValue" CASCADE
;

DROP TYPE IF EXISTS "ExternalReferenceType" CASCADE
;

DROP TYPE IF EXISTS "ExternalObjectReferenceType" CASCADE
;

/* Create ENUM Types */

CREATE TYPE "ThemeLayerValue" AS ENUM (
 'Virtual',
 'Physical',
 'Tags',
 'Unknown'
)
;

CREATE TYPE "TopoExpressionValue" AS ENUM (
 'Contains',
 'Overlaps',
 'Equals',
 'Within',
 'Crosses',
 'Other'
)
;
```

```
/* Create composite Types (UML DataTypes for external references) */

CREATE TYPE "ExternalObjectReferenceType" AS (
 "Name" varchar(100),
 "Uri" text
)
;

CREATE TYPE "ExternalReferenceType" AS (
 "ExternalObject" "ExternalObjectReferenceType",
 "InformationSystem" text
)
;

/* Create Tables */

CREATE TABLE "CellBoundary"
(
 "cellBoundaryGeom_geometry1D" geometry NULL,
 "cellBoundaryGeom_geometry2D" geometry NULL,
 "externalReference" "ExternalReferenceType" NULL,
 "Isvirtual" boolean NULL,
 "CellBoundaryID" varchar(100) NOT NULL,
 bounds varchar(100) NULL,
 "PrimalSpaceLayerID" varchar(100) NULL,
 CONSTRAINT "chk_CellBoundary_geom_xor" CHECK (
  NOT ("cellBoundaryGeom_geometry1D" IS NOT NULL AND "cellBoundaryGeom_geometry2D" IS NOT
NULL)
 )
)
;

CREATE TABLE "CellSpace"
(
 "cellSpaceGeom_geometry2D" geometry NULL,
 "cellSpaceGeom_geometry3D" geometry NULL,
 "CellSpaceName" varchar(100) NULL,
 "externalReference" "ExternalReferenceType" NULL,
 "Level" varchar(100) NULL,
 "Poi" boolean NULL,
 "CellSpaceID" varchar(100) NOT NULL,
 "PrimalSpaceLayerID" varchar(100) NOT NULL,
 "duality" varchar(100) NULL,
 CONSTRAINT "chk_CellSpace_geom_xor" CHECK (
  NOT ("cellSpaceGeom_geometry2D" IS NOT NULL AND "cellSpaceGeom_geometry3D" IS NOT NULL)
 )
)
;

CREATE TABLE "DualSpaceLayer"
(
 "Creationdate" timestamp without time zone NULL,
 "Terminationdate" timestamp without time zone NULL,
 "Islogical" boolean NULL,
 "Isdirected" boolean NULL,
 "DualSpaceLayerID" varchar(100) NOT NULL,
 "ThematicLayerID" varchar(100) NULL
)
;

CREATE TABLE "Edge"
(
 "Geometry" geometry NULL,
```

```
 "Weight" real NULL,
 "EdgeID" varchar(100) NOT NULL,
 "duality" varchar(100) NULL,
 "DualSpaceLayerID" varchar(100) NULL,
 connects jsonb NULL
)
;

CREATE TABLE "IndoorFeatures"
(
 "IndoorFeaturesID" varchar(100) NOT NULL,
 "layers" varchar(100) NULL  -- Reference to layers if needed, though usually layers point back to
IndoorFeatures
)
;

CREATE TABLE "InterLayerConnection"
(
 "Typeoftopoexpression" "TopoExpressionValue" NULL,
 "Comment" text NULL,
 "InterLayerConnectionID" varchar(100) NOT NULL,
 "IndoorFeaturesID" varchar(100) NULL
)
;

CREATE TABLE "InterLayerConnection_CellSpace"
(
 "connectedCells" varchar(100) NULL,
 "InterLayerConnectionID" varchar(100) NULL
)
;

CREATE TABLE "InterLayerConnection_Node"
(
 "connectedNodes" varchar(100) NULL,
 "InterLayerConnectionID" varchar(100) NULL
)
;

CREATE TABLE "InterLayerConnection_ThematicLayer"
(
 "connectedLayers" varchar(100) NULL,
 "InterLayerConnectionID" varchar(100) NULL
)
;

CREATE TABLE "Node"
(
 "Geometry" geometry NULL,
 "NodeID" varchar(100) NOT NULL,
 "duality" varchar(100) NULL,
 "DualSpaceLayerID" varchar(100) NOT NULL,
 connects jsonb NULL
)
;

CREATE TABLE "PrimalSpaceLayer"
(
 "Creationdate" timestamp without time zone NULL,
 "Terminationdate" timestamp without time zone NULL,
 "PrimalSpaceLayerID" varchar(100) NOT NULL,
 "ThematicLayerID" varchar(100) NULL
)
```

```
;

CREATE TABLE "ThematicLayer"
(
 "Semanticextension" boolean NULL,
 "Theme" "ThemeLayerValue" NULL,
 "ThematicLayerID" varchar(100) NOT NULL,
 "IndoorFeaturesID" varchar(100) NOT NULL
)
;

/* Create Primary Keys, Indexes, Uniques, Checks */

ALTER TABLE "CellBoundary" ADD CONSTRAINT "PK_CellBoundary"
 PRIMARY KEY ("CellBoundaryID")
;

ALTER TABLE "CellSpace" ADD CONSTRAINT "PK_CellSpace"
 PRIMARY KEY ("CellSpaceID")
;

ALTER TABLE "DualSpaceLayer" ADD CONSTRAINT "PK_DualSpaceLayer"
 PRIMARY KEY ("DualSpaceLayerID")
;

ALTER TABLE "Edge" ADD CONSTRAINT "PK_Edge"
 PRIMARY KEY ("EdgeID")
;

ALTER TABLE "IndoorFeatures" ADD CONSTRAINT "PK_IndoorFeatures"
 PRIMARY KEY ("IndoorFeaturesID")
;

ALTER TABLE "InterLayerConnection" ADD CONSTRAINT "PK_InterLayerConnection"
 PRIMARY KEY ("InterLayerConnectionID")
;

ALTER TABLE "Node" ADD CONSTRAINT "PK_Node"
 PRIMARY KEY ("NodeID")
;

ALTER TABLE "PrimalSpaceLayer" ADD CONSTRAINT "PK_PrimalSpaceLayer"
 PRIMARY KEY ("PrimalSpaceLayerID")
;

ALTER TABLE "ThematicLayer" ADD CONSTRAINT "PK_ThematicLayer"
 PRIMARY KEY ("ThematicLayerID")
;

/* Create Foreign Key Constraints */

ALTER TABLE "CellBoundary" ADD CONSTRAINT "FK_CellBoundary_boundedBy"
 FOREIGN KEY (bounds) REFERENCES "CellSpace" ("CellSpaceID") ON DELETE No Action ON UPDATE No
Action
;

ALTER TABLE "CellBoundary" ADD CONSTRAINT "FK_CellBoundary_cellBoundaryMember"
 FOREIGN KEY ("PrimalSpaceLayerID") REFERENCES "PrimalSpaceLayer" ("PrimalSpaceLayerID") ON
DELETE No Action ON UPDATE No Action
;

ALTER TABLE "CellSpace" ADD CONSTRAINT "FK_CellSpace_cellSpaceMember"
```

```
 FOREIGN KEY ("PrimalSpaceLayerID") REFERENCES "PrimalSpaceLayer" ("PrimalSpaceLayerID") ON
DELETE No Action ON UPDATE No Action
;

ALTER TABLE "CellSpace" ADD CONSTRAINT "FK_CellSpace_duality"
 FOREIGN KEY ("duality") REFERENCES "Node" ("NodeID") ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "DualSpaceLayer" ADD CONSTRAINT "FK_DualSpaceLayer_dualSpace"
 FOREIGN KEY ("ThematicLayerID") REFERENCES "ThematicLayer" ("ThematicLayerID") ON DELETE No
Action ON UPDATE No Action
;

ALTER TABLE "Edge" ADD CONSTRAINT "FK_Edge_duality"
 FOREIGN KEY ("duality") REFERENCES "CellBoundary" ("CellBoundaryID") ON DELETE No Action ON
UPDATE No Action
;

ALTER TABLE "Edge" ADD CONSTRAINT "FK_Edge_DualSpaceLayer"
 FOREIGN KEY ("DualSpaceLayerID") REFERENCES "DualSpaceLayer" ("DualSpaceLayerID") ON DELETE
No Action ON UPDATE No Action
;

ALTER TABLE "InterLayerConnection" ADD CONSTRAINT "FK_InterLayerConnection_layerConnections"
 FOREIGN KEY ("IndoorFeaturesID") REFERENCES "IndoorFeatures" ("IndoorFeaturesID") ON DELETE No
Action ON UPDATE No Action
;

ALTER TABLE "InterLayerConnection_CellSpace" ADD CONSTRAINT "FK_InterLayerConnection_CellSpace_
connectedCells"
 FOREIGN KEY ("connectedCells") REFERENCES "CellSpace" ("CellSpaceID") ON DELETE No Action ON
UPDATE No Action
;

ALTER TABLE "InterLayerConnection_CellSpace" ADD CONSTRAINT "FK_InterLayerConnection_CellSpace_
InterLayerConnection"
 FOREIGN KEY ("InterLayerConnectionID") REFERENCES "InterLayerConnection"
("InterLayerConnectionID") ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "InterLayerConnection_Node" ADD CONSTRAINT "FK_InterLayerConnection_Node_
connectedNodes"
 FOREIGN KEY ("connectedNodes") REFERENCES "Node" ("NodeID") ON DELETE No Action ON UPDATE
No Action
;

ALTER TABLE "InterLayerConnection_Node" ADD CONSTRAINT "FK_InterLayerConnection_Node_
InterLayerConnection"
 FOREIGN KEY ("InterLayerConnectionID") REFERENCES "InterLayerConnection"
("InterLayerConnectionID") ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "InterLayerConnection_ThematicLayer" ADD CONSTRAINT "FK_InterLayerConnection_
ThematicLayer_connectedLayers"
 FOREIGN KEY ("connectedLayers") REFERENCES "ThematicLayer" ("ThematicLayerID") ON DELETE No
Action ON UPDATE No Action
;

ALTER TABLE "InterLayerConnection_ThematicLayer" ADD CONSTRAINT "FK_InterLayerConnection_
ThematicLayer_InterLayerConnection"
 FOREIGN KEY ("InterLayerConnectionID") REFERENCES "InterLayerConnection"
("InterLayerConnectionID") ON DELETE No Action ON UPDATE No Action
;
```

```
ALTER TABLE "Node" ADD CONSTRAINT "FK_Node_duality"
 FOREIGN KEY ("duality") REFERENCES "CellSpace" ("CellSpaceID") ON DELETE No Action ON UPDATE No
Action
;

ALTER TABLE "Node" ADD CONSTRAINT "FK_Node_DualSpaceLayer"
 FOREIGN KEY ("DualSpaceLayerID") REFERENCES "DualSpaceLayer" ("DualSpaceLayerID") ON DELETE
No Action ON UPDATE No Action
;

ALTER TABLE "PrimalSpaceLayer" ADD CONSTRAINT "FK_PrimalSpaceLayer_primalSpace"
 FOREIGN KEY ("ThematicLayerID") REFERENCES "ThematicLayer" ("ThematicLayerID") ON DELETE No
Action ON UPDATE No Action
;

ALTER TABLE "ThematicLayer" ADD CONSTRAINT "FK_ThematicLayer_layers"
 FOREIGN KEY ("IndoorFeaturesID") REFERENCES "IndoorFeatures" ("IndoorFeaturesID") ON DELETE No
Action ON UPDATE No Action
;

/* Create Table Comments, Sequences for Autonumber Columns */

COMMENT ON TABLE "InterLayerConnection"
 IS 'Describes the connection between two thematic layers. '
;

COMMENT ON COLUMN "InterLayerConnection"."Typeoftopoexpression"
 IS 'Describes the topological relationship between two layers (e.g. overlaps, contains, etc.).'
;

COMMENT ON TABLE "ThematicLayer"
 IS 'A layer of specific theme aggregating a primal and/or a dual space of a given environment.'
;

COMMENT ON COLUMN "ThematicLayer"."Semanticextension"
 IS 'Indicates whether semantic information is associated (true) or not (false) to the primal space of the
thematic layer.'
;

COMMENT ON COLUMN "ThematicLayer"."Theme"
 IS 'Determines the theme of the layer (e.g topographic, logical, etc.).'
;
```

Listing B.1

B.2. SQL Schema of IndoorGML Navigation

```
/* ---------------------------------------------------- */
/*  IndoorGML Navigation Schema Extension               */
/*  Apply AFTER IndoorGML_core.sql on the same database */
/* ---------------------------------------------------- */

/* Drop Navigation Tables */

DROP TABLE IF EXISTS "TransferSpace" CASCADE
;
```

```
DROP TABLE IF EXISTS "GeneralSpace" CASCADE
;

DROP TABLE IF EXISTS "ObjectSpace" CASCADE
;

DROP TABLE IF EXISTS "NonNavigableSpace" CASCADE
;

DROP TABLE IF EXISTS "NavigableBoundary" CASCADE
;

DROP TABLE IF EXISTS "NonNavigableBoundary" CASCADE
;

DROP TABLE IF EXISTS "NavigableSpace" CASCADE
;

DROP TABLE IF EXISTS "Route" CASCADE
;

DROP TABLE IF EXISTS "GeneralSpaceFunctionType" CASCADE
;

DROP TABLE IF EXISTS "LocomotionAccessType" CASCADE
;

DROP TABLE IF EXISTS "NavigableBoundaryFunctionType" CASCADE
;

DROP TABLE IF EXISTS "NetworkType" CASCADE
;

DROP TABLE IF EXISTS "TransferSpaceFunctionType" CASCADE
;

DROP TABLE IF EXISTS "TransferSpaceType" CASCADE
;

/* Drop ENUM Types */

DROP TYPE IF EXISTS "NavigableBoundaryFunctionType" CASCADE
;

DROP TYPE IF EXISTS "GeneralSpaceFunctionType" CASCADE
;

DROP TYPE IF EXISTS "LocomotionAccessType" CASCADE
;

DROP TYPE IF EXISTS "TransferSpaceCategoryType" CASCADE
;

DROP TYPE IF EXISTS "TransferSpaceFunctionType" CASCADE
;

/* Create ENUM Types */

CREATE TYPE "GeneralSpaceFunctionType" AS ENUM (
 'Administration',
 'Business, trade',
 'Education, training',
```

```
 'Recreation',
 'Laboratory',
 'Storage',
 'Security'
)
;

CREATE TYPE "LocomotionAccessType" AS ENUM (
 'Walking',
 'Flying',
 'Rolling',
 'Unspecified'
)
;

CREATE TYPE "NavigableBoundaryFunctionType" AS ENUM (
 'AnchorBoundary',
 'ConnectionBoundary'
)
;

CREATE TYPE "TransferSpaceFunctionType" AS ENUM (
 'ConnectionSpace',
 'AnchorSpace'
)
;

CREATE TYPE "TransferSpaceCategoryType" AS ENUM (
 'Door',
 'Window'
)
;

/* Create Navigation Tables */

CREATE TABLE "NavigableSpace"
(
 "Locomotiontype" "LocomotionAccessType" NULL,
 "NavigableSpaceID" varchar(100) NOT NULL
)
;

CREATE TABLE "NonNavigableSpace"
(
 "NonNavigableSpaceID" varchar(100) NOT NULL
)
;

CREATE TABLE "GeneralSpace"
(
 "Function" "GeneralSpaceFunctionType" NULL,
 "GeneralSpaceID" varchar(100) NOT NULL
)
;

CREATE TABLE "TransferSpace"
(
 "Function" "TransferSpaceFunctionType" NULL,
 "Type" "TransferSpaceCategoryType" NULL,
 "TransferSpaceID" varchar(100) NOT NULL
)
;
```

```
CREATE TABLE "ObjectSpace"
(
 "Description" text NULL,
 "Containedfeatures" varchar(100) NULL,
 "ObjectSpaceID" varchar(100) NOT NULL
)
;

CREATE TABLE "NavigableBoundary"
(
 "NavigableBoundaryFunction" "NavigableBoundaryFunctionType" NULL,
 "Boundaryorientation" boolean NULL,
 "NavigableBoundaryID" varchar(100) NOT NULL
)
;

CREATE TABLE "NonNavigableBoundary"
(
 "NonNavigableBoundaryID" varchar(100) NOT NULL
)
;

CREATE TABLE "Route"
(
 "Creationdate" timestamp without time zone NULL,
 "RouteID" varchar(100) NOT NULL,
 "routeNode" jsonb NULL,
 "routeEdge" jsonb NULL
)
;

/* Primary Keys */

ALTER TABLE "NavigableSpace" ADD CONSTRAINT "PK_NavigableSpace"
 PRIMARY KEY ("NavigableSpaceID")
;

ALTER TABLE "NonNavigableSpace" ADD CONSTRAINT "PK_NonNavigableSpace"
 PRIMARY KEY ("NonNavigableSpaceID")
;

ALTER TABLE "GeneralSpace" ADD CONSTRAINT "PK_GeneralSpace"
 PRIMARY KEY ("GeneralSpaceID")
;

ALTER TABLE "TransferSpace" ADD CONSTRAINT "PK_TransferSpace"
 PRIMARY KEY ("TransferSpaceID")
;

ALTER TABLE "ObjectSpace" ADD CONSTRAINT "PK_ObjectSpace"
 PRIMARY KEY ("ObjectSpaceID")
;

ALTER TABLE "NavigableBoundary" ADD CONSTRAINT "PK_NavigableBoundary"
 PRIMARY KEY ("NavigableBoundaryID")
;

ALTER TABLE "NonNavigableBoundary" ADD CONSTRAINT "PK_NonNavigableBoundary"
 PRIMARY KEY ("NonNavigableBoundaryID")
;

ALTER TABLE "Route" ADD CONSTRAINT "PK_Route"
 PRIMARY KEY ("RouteID")
```

```
;

/* Foreign Keys */

ALTER TABLE "NavigableSpace" ADD CONSTRAINT "FK_NavigableSpace_CellSpace"
 FOREIGN KEY ("NavigableSpaceID") REFERENCES "CellSpace" ("CellSpaceID") ON DELETE No Action ON
UPDATE No Action
;

ALTER TABLE "NonNavigableSpace" ADD CONSTRAINT "FK_NonNavigableSpace_CellSpace"
 FOREIGN KEY ("NonNavigableSpaceID") REFERENCES "CellSpace" ("CellSpaceID") ON DELETE No Action
ON UPDATE No Action
;

ALTER TABLE "GeneralSpace" ADD CONSTRAINT "FK_GeneralSpace_NavigableSpace"
 FOREIGN KEY ("GeneralSpaceID") REFERENCES "NavigableSpace" ("NavigableSpaceID") ON DELETE No
Action ON UPDATE No Action
;

ALTER TABLE "TransferSpace" ADD CONSTRAINT "FK_TransferSpace_NavigableSpace"
 FOREIGN KEY ("TransferSpaceID") REFERENCES "NavigableSpace" ("NavigableSpaceID") ON DELETE No
Action ON UPDATE No Action
;

ALTER TABLE "ObjectSpace" ADD CONSTRAINT "FK_ObjectSpace_NonNavigableSpace"
 FOREIGN KEY ("ObjectSpaceID") REFERENCES "NonNavigableSpace" ("NonNavigableSpaceID") ON
DELETE No Action ON UPDATE No Action
;

ALTER TABLE "NavigableBoundary" ADD CONSTRAINT "FK_NavigableBoundary_CellBoundary"
 FOREIGN KEY ("NavigableBoundaryID") REFERENCES "CellBoundary" ("CellBoundaryID") ON DELETE No
Action ON UPDATE No Action
;

ALTER TABLE "NonNavigableBoundary" ADD CONSTRAINT "FK_NonNavigableBoundary_CellBoundary"
 FOREIGN KEY ("NonNavigableBoundaryID") REFERENCES "CellBoundary" ("CellBoundaryID") ON DELETE
 No Action ON UPDATE No Action
;
```

Listing B.2

C

ANNEX C (INFORMATIVE) REVISION HISTORY

ANNEX C (INFORMATIVE) REVISION HISTORY

DATE	RELEASE	EDITOR	PRIMARY CLAUSES MODIFIED	DESCRIPTION
2026-06-17	0.1	Zhiyong Wang	all	initial version derived from SQL generation workflow document
2026-06-18	0.2	Zhiyong Wang	5, 6, 7, 8, annex B	align schema with ENUM types, jsonb connects/route arrays, and EA table naming
2026-07-02	0.3	Zhiyong Wang	5, 6	add Part 2c-specific symbols and abbreviated terms table (clause 5); replace vague figure references with numbered cross-references in clause 6.1.1 and remove redundant follow-up sentences; rebuild document.pdf and document.html
2026-07-30	0.4	Zhiyong Wang	5, 6, 7	flatten geometry DataTypes to path-named PostGIS columns; encode ExternalReference DataTypes as PostgreSQL composite types; PascalCase table names; edition 0.0.9

BIBLIOGRAPHY

BIBLIOGRAPHY

[1] H. Butler, M. Daly, A. Doyle, S. Gillies, S. Hagen, T. Schaub, Internet Engineering Task Force (IETF): IETF RFC 7946 (PROPOSED STANDARD), *The GeoJSON Format*. RFC Publisher (2016). https://www.rfc-editor.org/info/rfc7946.

[2] Internet Engineering Task Force (IETF): IETF RFC 8259 (INTERNET STANDARD), *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC Publisher (2017). https://www.rfc-editor.org/info/rfc8259.

[3] The PostGIS Development Group: **PostGIS Manual,** https://postgis.net/docs/

[4] Ben-Kiki, O., Evans, C., Ingy döt Net: **YAML Ain't Markup Language,** https://yaml.org/